Investment Tracker — Database Schema

A relational schema (works in Postgres/MySQL/SQLite) designed to support:

- XIRR — overall portfolio and per-stock
- Net worth over time
- Realized profit (FIFO lot-based)
- Unrealized profit (mark-to-market)
- Dividend/corporate action tracking
- Multi-account, multi-asset-class support

---

1. `accounts`

Broker/demat/mutual-fund accounts you invest through.

Column	Type	Notes
account_id	PK, INT/UUID	
account_name	VARCHAR	e.g. "Zerodha", "ICICI Direct"
broker_name	VARCHAR	
account_type	ENUM	demat, mutual_fund, pf, nps, crypto_exchange, bank
currency	VARCHAR(3)	INR, USD, etc.
created_at	DATE	

---

2. `assets`

Master list of every security you've ever held (the "stock master").

Column	Type	Notes
asset_id	PK	
symbol	VARCHAR	Ticker, e.g. RELIANCE
isin	VARCHAR	Unique security identifier
name	VARCHAR	Full name
asset_type	ENUM	stock, etf, mutual_fund, bond, fd, crypto, gold
exchange	VARCHAR	NSE, BSE, NASDAQ, etc.
sector	VARCHAR	For allocation reports
currency	VARCHAR(3)	

---

3. `transactions`

Every buy, sell, dividend, fee — the source of truth for everything else.

Column	Type	Notes
transaction_id	PK	
account_id	FK → accounts	
asset_id	FK → assets	
transaction_type	ENUM	buy, sell, dividend, bonus, split, interest, fee, deposit, withdrawal
transaction_date	DATE	
quantity	DECIMAL	Null for cash-only entries
price_per_unit	DECIMAL	
total_amount	DECIMAL	quantity × price ± charges
fees	DECIMAL	Brokerage, STT
taxes	DECIMAL	
notes	TEXT	

4. `lots` (FIFO cost-basis tracking)

Each buy transaction creates a lot. Needed for accurate realized P&L and long/short-term tax classification.

Column	Type	Notes
lot_id	PK	
account_id	FK → accounts	
asset_id	FK → assets	
buy_transaction_id	FK → transactions	
buy_date	DATE	
quantity_original	DECIMAL	
quantity_remaining	DECIMAL	Decremented as sold
cost_per_unit	DECIMAL	Includes fees, adjusted for splits/bonus

5. `lot_sales`

Maps a sell transaction to the lot(s) it consumed (FIFO or specific-lot).

Column	Type	Notes
lot_sale_id	PK	
lot_id	FK → lots	
sell_transaction_id	FK → transactions	
quantity_sold	DECIMAL	
sale_price_per_unit	DECIMAL	
cost_basis	DECIMAL	quantity_sold × lot.cost_per_unit
realized_pnl	DECIMAL	(sale − cost_basis)
holding_period_days	INT	sale_date − buy_date, for tax bucket

Realized profit = `SUM(realized_pnl)` from this table, filterable by asset/account/date range.

---

6. `price_history`

Daily (or periodic) closing prices — needed for unrealized P&L and net worth over time.

Column	Type	Notes
price_id	PK	
asset_id	FK → assets	
price_date	DATE	
close_price	DECIMAL	
source	VARCHAR	API/manual

Unique constraint on `(asset_id, price_date)`.

Unrealized profit (per stock) = `quantity_remaining × (latest close_price − cost_per_unit)` summed across open lots.

---

7. `dividends`

Separate from transactions for easy reporting (yield, tax withheld, etc.) — optionally also mirrored as a `transactions` row.

Column	Type	Notes
dividend_id	PK	
asset_id	FK → assets	
account_id	FK → accounts	
ex_date	DATE	
pay_date	DATE	
amount_per_share	DECIMAL	
total_amount	DECIMAL	
tax_withheld	DECIMAL	

8. `corporate_actions`

Splits, bonuses, mergers — used to retroactively adjust `lots.cost_per_unit` and quantities.

Column	Type	Notes
action_id	PK	
asset_id	FK → assets	
action_type	ENUM	split, bonus, merger, spinoff, name_change
action_date	DATE	
ratio	VARCHAR	e.g. "1:2" for a bonus
notes	TEXT	

9. `cash_flows`

The dedicated table that makes XIRR computation clean — every external cash movement, tagged by scope. Investments show as negative flows, withdrawals/current value as positive.

Column	Type	Notes
cash_flow_id	PK	
scope_type	ENUM	portfolio, account, asset
scope_id	INT	account_id or asset_id depending on scope_type; null for portfolio-wide
flow_date	DATE	
amount	DECIMAL	Negative = money in (buy/deposit), Positive = money out (sell/withdrawal/dividend)
flow_type	VARCHAR	buy, sell, dividend, deposit, withdrawal, mark_to_market

XIRR (per stock): pull all `cash_flows` where `scope_type='asset', scope_id=<asset_id>`, append a final synthetic positive flow = current market value as of today, then run the XIRR algorithm (Newton-Raphson on the NPV function).

XIRR (overall portfolio): same, with `scope_type='portfolio'`, aggregating every buy/sell/dividend across all accounts plus one final flow = total current net worth.

This table can be **populated automatically via triggers/ETL from `transactions`**, so you don't hand-maintain it.

---

10. `networth_snapshots`

Point-in-time portfolio value — one row per day/week, built by a scheduled job.

Column	Type	Notes
snapshot_id	PK	
snapshot_date	DATE	Unique
total_invested	DECIMAL	Cumulative capital deployed (net of withdrawals)
total_current_value	DECIMAL	Sum of quantity_remaining × close_price
cash_balance	DECIMAL	Uninvested cash
total_realized_pnl	DECIMAL	Cumulative to date
total_unrealized_pnl	DECIMAL	current_value − cost_basis of open lots
net_worth	DECIMAL	current_value + cash_balance

---

11. `networth_by_asset_class` (snapshot breakdown)

Column	Type	Notes
id	PK	
snapshot_id	FK → networth_snapshots	
asset_class	VARCHAR	stock, mf, gold, crypto, fd, cash
value	DECIMAL	

Lets you chart allocation drift over time without re-deriving it each time.

---

12. `benchmarks` (optional, for alpha/comparison)

Column	Type	Notes
id	PK	
benchmark_name	VARCHAR	Nifty 50, S&P 500
price_date	DATE	
close_value	DECIMAL	

Compare your XIRR against benchmark CAGR over the same cash-flow-weighted period.

---

13. `xirr_cache` (optional, performance)

XIRR is iterative/expensive — cache results instead of recomputing on every page load.

Column	Type	Notes
id	PK	
scope_type	ENUM	portfolio, account, asset
scope_id	INT	nullable
as_of_date	DATE	
xirr_value	DECIMAL	
computed_at	TIMESTAMP	

How the pieces connect

Key derived metrics (all computable from the above, no extra tables needed)

Metric	Formula
Realized P&L	`SUM(lot_sales.realized_pnl)`
Unrealized P&L	`SUM(open_lots.quantity_remaining × (latest_price − cost_per_unit))`
Per-stock XIRR	XIRR of `cash_flows WHERE scope_type='asset'` + current value as final flow
Overall XIRR	XIRR of `cash_flows WHERE scope_type='portfolio'` + net worth as final flow
Net worth over time	Time series from `networth_snapshots.net_worth`
Asset allocation	Latest `networth_by_asset_class` rows
CAGR vs benchmark	Compare portfolio XIRR to `benchmarks` return over same window

Implementation notes

- Store money as `DECIMAL(18,4)`, never FLOAT — avoids rounding drift over years of compounding.
- Populate `cash_flows` and `networth_snapshots` via a nightly job (or trigger) off `transactions` + `price_history`, rather than hand-entering them.
- Index `transactions(asset_id, transaction_date)`, `price_history(asset_id, price_date)`, and `cash_flows(scope_type, scope_id, flow_date)` — these are your hot query paths.
- For XIRR, most stacks use a Newton-Raphson solver (e.g. `pyxirr` in Python, or a small custom function in JS) rather than a DB-native function.